



**INSTITUTO POLITÉCNICO
DO CÁVADO E DO AVE**



**ESCOLA
SUPERIOR
DE TECNOLOGIA
IPCA**

MANIPULAÇÃO DE ESTRUTURAS DE DADOS DINÂMICAS

Nuno Miguel Fernandes Oliveira

Nº 31550 – Regime Pós-laboral

Ano letivo 2024/2025

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

Índice

Índice de Figuras.....	3
Resumo.....	4
1. Introdução	5
1.1. Contexto do Projeto	5
1.2. Interpretação do problema	5
1.3. Objetivos.....	5
1.4. Metodologia.....	5
2. Enquadramento Teórico.....	6
2.1. Estruturas	6
2.2. Listas Ligadas Simples.....	6
2.3. Funções.....	7
2.3.1. Criar Antena	7
2.3.2. Inserir Antena	7
2.3.3. Criar Zonas Nefastas.....	7
2.3.4. Remover Antena	7
2.3.5. Ler Ficheiro	7
2.3.6. Gravar Ficheiro	8
2.3.7. Ler Listas Binário.....	8
2.3.8. Guardar Listas Binário	8
2.3.9. Mostrar Matriz	8
3. Implementação	9
3.1. Percurso até à solução	9
3.1.1. Escolha do Método Aplicado	9
3.1.2. Descoberta do Algoritmo.....	9
3.2. Algoritmo na Linguagem C	10
3.3. Mostrar Matriz na Linguagem C.....	11
3.4. Guardar Listas e Ler Listas em Ficheiro Binário	12
Conclusão	13

Índice de Figuras

Figura I	5
Figura II	6
Figura III	6
Figura IV	9
Figura V	10
Figura VII	10
Figura VI	10
Figura VIII	11
Figura IX	11
Figura X	11
Figura XI	12

Resumo

O projeto *Manipulação de Estruturas de Dados Dinâmicas* visa a prática e a consolidação dos conhecimentos relativos à definição e à manipulação de estruturas de dados dinâmicas na linguagem de programação C.

Considerou-se uma cidade com um número indefinido de antenas. Quando duas antenas têm a mesma frequência ocorrem efeitos nefastos. Deste modo, o objetivo é solucionar e descobrir a localização onde os efeitos nefastos são sentidos, caso ocorram. Dado o problema noticiado, a solução foi desenvolvida com o recurso ao método de listas ligadas simples e apresentada em formato de matriz.

1. Introdução

1.1. Contexto do Projeto

No âmbito da unidade curricular Estruturas de Dados Avançadas, do 2º semestre, da Licenciatura em Engenharia de Sistemas Informáticos, desenvolveu-se um projeto com o objetivo de concretizar os conhecimentos no que toca à manipulação de estruturas de dados dinâmicas.

1.2. Interpretação do problema

Considera-se uma cidade com um número indefinido de antenas. Cada antena é sintonizada numa frequência específica, representada por um carácter.

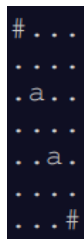


Figura I

O efeito nefasto ocorre em qualquer localização que esteja perfeitamente alinhada com duas antenas da mesma frequência. Assim, tem-se que, para qualquer par de antenas com a mesma frequência, existem duas localizações nefastas, representadas pelos caracteres '#', uma de cada lado das antenas, como ilustrado na *Figura I*. Antenas com frequências diferentes não criam localizações com efeito nefasto.

1.3. Objetivos

Quando ocorrem zonas nefastas, resultantes da presença de duas antenas a operar na mesma frequência, será necessário calcular as coordenadas exatas onde as mesmas se encontram. Em virtude de tal necessidade e com base na análise das antenas de uma determinada região, é essencial a descoberta do algoritmo para o cálculo automatizado das zonas nefastas.

1.4. Metodologia

Com o objetivo de assegurar o cumprimento dos objetivos, desenvolveu-se uma estrutura de dados adequada ao problema. Esta estrutura de dados deve permitir o armazenamento e a manipulação das informações relativas às antenas, nomeadamente as suas localizações e frequências. O método listas ligadas revelou-se competente e apropriado para a realização dos objetivos.

2. Enquadramento Teórico

2.1. Estruturas

Estruturas são utilizadas para representar entidades que possuem mais do que um parâmetro, permitindo a sua caracterização e diferenciação.

As antenas necessitam de mais do que um parâmetro para a sua diferenciação e para o cálculo das zonas nefastas. Cada antena é representada por uma estrutura composta por estes parâmetros. Assim sendo, pode-se afirmar que cada antena é uma estrutura.

Os parâmetros indispensáveis para que seja possível a diferenciação entre si, são: a frequência, a linha e a coluna, uma vez que o produto final será visualizado em forma matricial. Considerando que, a metodologia usada se baseia em Listas Ligadas Simples, será imperativo que a estrutura contenha um parâmetro adicional, o qual permita a ligação à antena seguinte, caso exista.

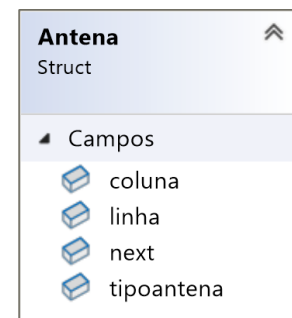


Figura II

A Figura II ilustra, graficamente, a estrutura adotada no desenvolvimento do projeto.

2.2. Listas Ligadas Simples

Conclui-se que o método das Listas Ligadas Simples é adequado para responder ao objetivo do projeto, uma vez que se trata de uma abordagem que permite a gestão de memória dinâmica, ou seja, o seu tamanho adapta-se às necessidades, o que permite uma melhor utilização dos recursos da memória. Salientam-se outras vantagens como a facilidade de inserção de um elemento em qualquer posição bem como a simplicidade de remoção de um elemento.

Este método assegura que todas as antenas permaneçam ligadas entre si, facilitando o seu armazenamento e a sua gestão. Contudo, será necessário um cuidado extremo na manipulação dos dados e das respetivas ligações, visto que um pequeno erro poderá resultar numa perda de informação.

O termo técnico usado para designar a ligação entre antenas é *apontador*.



Figura III

A Figura III ilustra graficamente como uma lista ligada simples opera.

2.3. Funções

2.3.1. Criar Antena

`Antena*` CriarAntena(`int` linha, `int` coluna, `char` tipoantena);

A função *CriarAntena* tem como propósito criar uma antena com base nos parâmetros fornecidos: a linha, a coluna e a frequência.

2.3.2. Inserir Antena

`Antena*` InserirAntena(`Antena*` head, `Antena*` nova);

A função *InserirAntena* tem como finalidade a inserção ordenada de uma nova antena na respetiva lista. Decidiu-se que o critério de ordenação é de ordem crescente, priorizando as linhas e, posteriormente, as colunas.

2.3.3. Criar Zonas Nefastas

`Antena*` CriarZonasNefastas(`Antena*` head);

A função *CriarZonasNefastas* visa verificar a existência de antenas que operem na mesma frequência. Caso essa situação seja confirmada, a função *CriarZonasNefastas* irá criar uma zona nefasta com base na localização das respetivas antenas.

2.3.4. Remover Antena

`bool` RemoverAntena(`Antena**` head, `int` linha, `int` coluna);

A função *RemoverAntena* tem como objetivo a remoção de uma antena da lista, com base nos parâmetros fornecidos: o início da lista, a linha e a coluna.

2.3.5. Ler Ficheiro

`bool` LerFicheiro(`const char*` NomeFicheiro, `Antena**` head);

A função *LerFicheiro* permite a leitura de um ficheiro não binário. O ficheiro deverá conter uma ilustração de um mapa com antenas. À medida que se lê as antenas, a função *LerFicheiro* invoca duas outras funções: *CriarAntena* e *InserirAntena*.

2.3.6. Gravar Ficheiro

```
bool GravarFicheiro(const char* NomeFicheiro, Antena* head);
```

A função *GravarFicheiro* possibilita a escrita em um ficheiro não binário. Pretende-se, com esta função, ilustrar um mapa com apenas as antenas na forma matricial.

2.3.7. Ler Listas Binário

```
bool LerListasBinario(const char* NomeFicheiro, Antena** head);
```

A função *LerListasBinario* consiste na leitura de um ficheiro binário. O ficheiro deverá conter uma lista das antenas. Conforme lê as antenas, a função *LerListasBinario* invoca duas outras funções: *CriarAntena* e *InserirAntena*.

2.3.8. Guardar Listas Binário

```
bool GuardarListasBinario(const char* NomeFicheiro, Antena* head);
```

A função *GuardarListasBinario* tem como propósito guardar a lista com as antenas num ficheiro binário.

2.3.9. Mostrar Matriz

```
bool MostrarMatriz(Antena* head, Antena* headnefasto);
```

A função *MostrarMatriz* facilita a ilustração do mapa com as antenas e com as zonas nefastas simultaneamente, no terminal, e na forma matricial.

3. Implementação

3.1. Percurso até à solução

3.1.1. Escolha do Método Aplicado

Inicialmente, admitiu-se que a melhor abordagem para a resolução do problema seria a criação de um mapa, na forma matricial, por meio de *Arrays*.

Rapidamente, compreendeu-se que não era viável para o problema em questão, uma vez que se pretendia uma solução totalmente dinâmica. Depois da análise do tema, entendeu-se que o método Listas Ligadas Simples teria o desempenho necessário para a resolução do problema.

3.1.2. Descoberta do Algoritmo

A resolução do problema depende, significativamente, do algoritmo utilizado, dado que diferentes algoritmos produzem resultados distintos, sendo, por isso, uma etapa crucial na resolução do problema.

O objetivo do presente problema passa pela descoberta da localização das zonas nefastas. Ao fim de auxiliar a verificação da eficácia do algoritmo implementado, elaborou-se uma fórmula matemática, *Figura IV*, cujo resultado corresponde ao número de zonas nefastas.

A fórmula para o cálculo do número de efeitos nefastos é a seguinte:

$$N = \sum A \times (A - 1)$$

N – Número de efeitos nefastos

A – Número de antenas com a mesma frequência

Figura IV

3.2. Algoritmo na Linguagem C

O algoritmo implementado no projeto encontra-se na função *CriarZonasNefastas*. A *Figura V* demonstra, na linguagem C, o algoritmo.

```
363 // Algoritmo para descoberta das coordenadas das zonas Nefastas
364 novo->next = NULL;
365 novo->linha = aux->linha + (aux->linha - aux2->linha);
366 novo->coluna = aux->coluna + (aux->coluna - aux2->coluna);
367 novo->tipoantena = '#';
```

Figura V

Tendo em conta o propósito da função *CriarZonasNefastas*, será necessário percorrer as listas das antenas, de forma a verificar a existência de duas ou mais antenas que partilhem a mesma frequência. Assim sendo, será essencial a criação de dois ciclos.

Os ciclos representados na *Figura VI* percorrem a lista das antenas, comparando cada uma com as restantes. Enquanto o apontador

aux se encontra na primeira posição da lista, o apontador *aux2* percorre as restantes posições (*Figura VII*, linha 385), garantindo que a comparação seja efetuada. Assim que, o apontador *aux2* atinja o final da lista, este é reposicionado no início (*Figura VII*, linha 387), ao passo que, o apontador *aux* passa a situar-se no elemento seguinte ao anteriormente considerado (*Figura VII*, linha 388). Este processo repete-se até o apontador *aux* atinja o final da lista das antenas.

```
350 while (aux != NULL) {
351     while (aux2 != NULL) {
```

Figura VII

```
385 aux2 = aux2->next;
386 }
387 aux2 = head;
388 aux = aux->next;
389 }
```

Figura VI

Durante o processo de comparação, sempre que se verifica a igualdade de frequência entre duas antenas, será criada uma zona nefasta. Dessarte, será possível determinar a localização exata da zona nefasta, recorrendo ao algoritmo aplicado na resolução deste problema.

O algoritmo possibilita a comparação da linha da antena apontada por *aux* com a linha da antena apontada por *aux2*, bem como a coluna da antena apontada por *aux* com a coluna da antena apontada por *aux2*. Deste modo, efetua-se um cálculo cujo resultado corresponde às coordenadas da localização da zona nefasta (*Figura V*, linha 365 e linha 366).

3.3. Mostrar Matriz na Linguagem C

Com o propósito que haja uma correta visualização dos dados no formato matricial, elaborou-se um algoritmo capaz de calcular o tamanho mínimo necessário de matriz, na função *MostrarMatriz*. Assim sendo, garantiu-se que todas as antenas e todas as zonas nefastas fossem corretamente apresentadas.

```

217 while (aux != NULL) {
218     if (aux->coluna > ncolunasmatriz)
219         ncolunasmatriz = aux->coluna;
220     if (aux->linha > nlinhasmatriz)
221         nlinhasmatriz = aux->linha;
222     aux = aux->next;
223 }
```

Figura VIII

A primeira parte do algoritmo tem como finalidade percorrer a lista das antenas e a lista das zonas nefastas,

permitindo a identificação dos valores mínimos e máximos das respetivas coordenadas (Figura VIII e Figura IX). Desta forma, determina-se o tamanho da matriz e procede-se à representação visual do mapa, com a colocação das antenas e das zonas nefastas nas respetivas posições. Neste momento, tem início a segunda fase do algoritmo.

```

224 while (nef != NULL) {
225     if (nef->coluna > ncolunasmatriz)
226         ncolunasmatriz = nef->coluna;
227     if (nef->coluna < ncolunasNEF)
228         ncolunasNEF = nef->coluna;
229     if (nef->linha > nlinhasmatriz)
230         nlinhasmatriz = nef->linha;
231     if (nef->linha < nlinhasNEF)
232         nlinhasNEF = nef->linha;
233     nef = nef->next;
234 }
```

Figura X

```

236 // Ciclo para criar as linhas. Inicia com o menor número até chegar ao maior número.
237 for (int i = nlinhasNEF; i <= nlinhasmatriz; i++) {
238
239     // Ciclo para criar as colunas. Inicia com o menor número até chegar ao maior número.
240     for (int j = ncolunasNEF; j <= ncolunasmatriz; j++) {
241         if (aux2 != NULL && i == aux2->linha && j == aux2->coluna) {
242             printf("%c", aux2->tipoantena);
243             aux2 = aux2->next;
244
245             // Se uma Antena e Nefasta tiverem as mesmas coordenadas, dar prioridade à impressão da Antena
246             if (nef2 != NULL && i == nef2->linha && j == nef2->coluna) {
247                 nef2 = nef2->next;
248             }
249
250             continue;
251         }
252         else if ((nef2 != NULL) && (i == nef2->linha) && (j == nef2->coluna)) {
253             printf("%c", nef2->tipoantena);
254             nef2 = nef2->next;
255             continue;
256         }
257     }
258 }
```

Figura IX

A matriz final é iniciada com as coordenadas retiradas da primeira fase do algoritmo (Figura IX, linha 237 e linha 240). A cada iteração, as variáveis *i* e *j* são incrementadas em uma unidade, nos seus respetivos ciclos, até atingirem os valores mais altos, retirados também na primeira fase do algoritmo. Caso os valores de ambas as variáveis coincidirem com as coordenadas de uma antena ou de uma zona nefasta, a frequência da antena é impressa para o terminal (Figura IX, linha 241 à linha 255). No caso particular de uma antena e uma zona nefasta partilharem exatamente as mesmas coordenadas, decidiu-se priorizar a impressão da antena sobre a zona de nefasta (Figura IX, linha 246 e linha 247).

3.4. Guardar Listas e Ler Listas em Ficheiro Binário

Após a criação de uma lista, será necessário um local onde seja possível guardar a informação que se recolhe do programa. Assim, decidiu-se guardar a informação num ficheiro binário. As vantagens do ficheiro binário são: a segurança na informação, o facto de ser difícil o utilizador modificar e/ou entender o conteúdo e a eficácia quer seja a guardar ou a ler informação do ficheiro.

As estruturas *Antena* possuem um campo que é um apontador. Realçando que, não se guardou essa estrutura em ficheiro binário, dado que poderia apontar para algo na memória que não se deseja, pelo que, se criou uma outra estrutura denominada por *AntenaFile*. A estrutura *AntenaFile* apresenta os mesmos campos que a estrutura *Antena*, exceto o campo apontador.

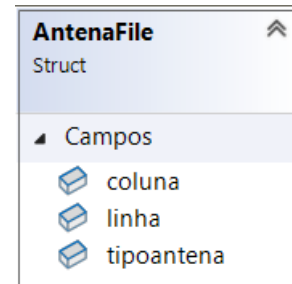


Figura XI

Com o intuito de guardar a informação contida nas listas num ficheiro binário, será necessário transferir os dados da estrutura *Antena* para a estrutura *AntenaFile*, procedendo-se, de seguida, à implementação da função *GuardarListasBinario*. Deste modo, os dados serão guardados, sequencialmente, um bloco de cada vez, ou seja, uma antena de cada vez.

A fim de ler a informação de uma lista previamente guardada em ficheiro binário, será fulcral transferir os dados da estrutura *AntenaFile* para a estrutura *Antena*, recorrendo-se à implementação da função *LerListasBinario*. Assim, os dados são lidos, sequencialmente, um bloco de cada vez, ou seja, uma antena de cada vez.

Conclusão

A solução apresentada expressa um desempenho correto e eficiente, sendo capaz de criar, ler e ordenar as antenas, as suas zonas nefastas bem como guardar as respetivas listas em ficheiros binários.

Em forma de conclusão, na primeira fase do projeto, da unidade curricular *Estrutura de Dados Avançadas*, alcançaram-se os objetivos pretendidos, proporcionando-se, assim, uma aplicação prática dos conceitos de manipulação de estruturas de dados dinâmicas para a resolução eficiente de problemas.

Link Repositório GitHub:

<https://github.com/a31550/ProjectEDA.git>